LAB 6

Code and Branch Coverage Analysis

Tennis Scoring Python Application

For this lab, I wrote separate pytest test files for statement coverage and branch coverage. I ran both test sets against the provided tennis scoring program and used coverage.py to check how much of the application was exercised.

Results

Analysis	Tests passed	Measured coverage	Result
Code (statement) coverage	25	38 / 38 statements	**100%**
Branch coverage	24	30 / 30 branches	**100%**

What I tested

The application has functions for tennis game scores, set winners, and match status. My tests include regular scores, deuce and advantage, both players winning, negative point inputs, sets that are still undecided, match wins, a decider, and a match that is still in progress.

How I ran the analysis

```
py -m coverage erase
py -m coverage run --source=tennis_scoring -m pytest -q CodeCoverageTest_34_tennis_scoring.py
py -m coverage report -m

py -m coverage erase
py -m coverage run --branch --source=tennis_scoring -m pytest -q BranchCoverageTest_34_tennis_scoring.py
py -m coverage report -m
```

Statement coverage checks whether executable statements ran. Branch coverage also checks the different outcomes of decisions. Both results are 100%, which is above the lab minimum of 90%.

Repository commits

I committed the code coverage tests and branch coverage tests separately, then committed the two result screenshots. Commits: `5d78ced` (code tests), `49d8b1f` (branch tests), and `be153b7` (screenshots). The work is pushed to my individual fork.

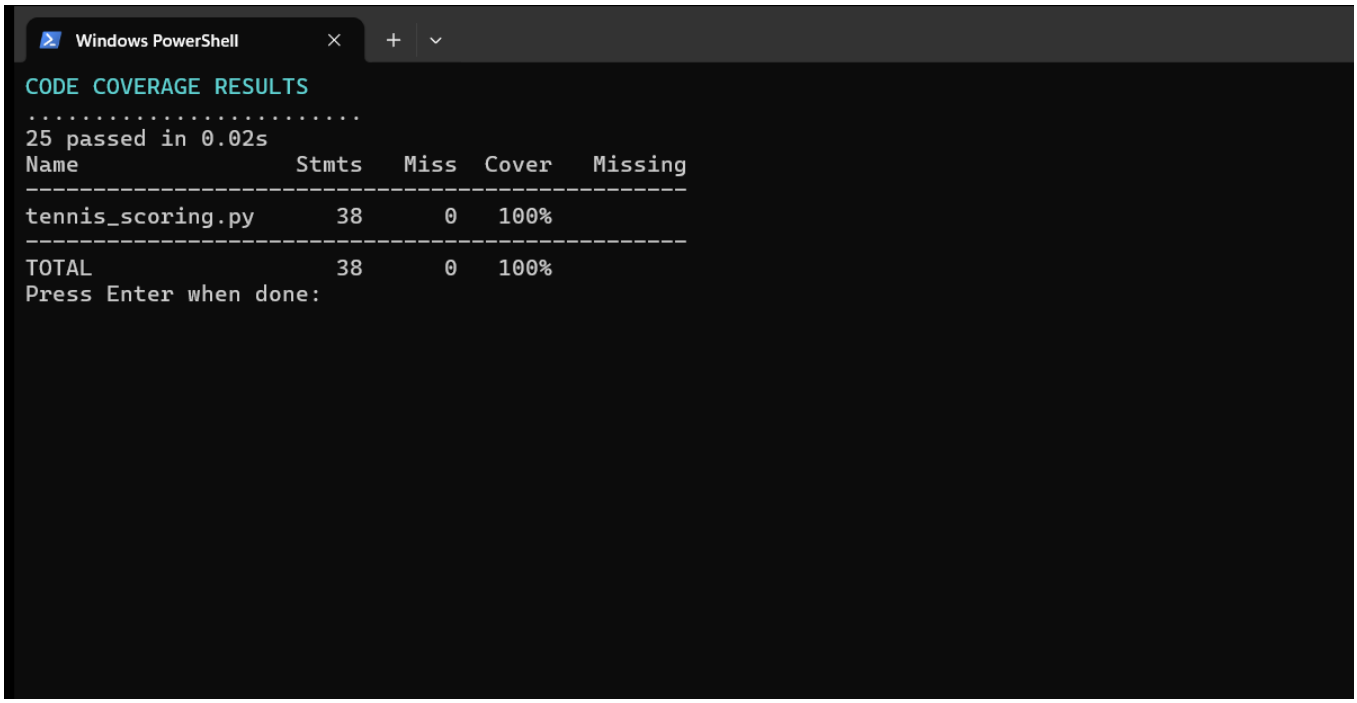
Conclusion

This lab helped me see why line coverage by itself is not enough. The branch tests check alternative outcomes too, and the final reports show that all measured statements and branches in the tennis scoring module were covered.

The following pages contain the terminal result screenshots and the two Python test files used for the analysis.

Code Coverage Result

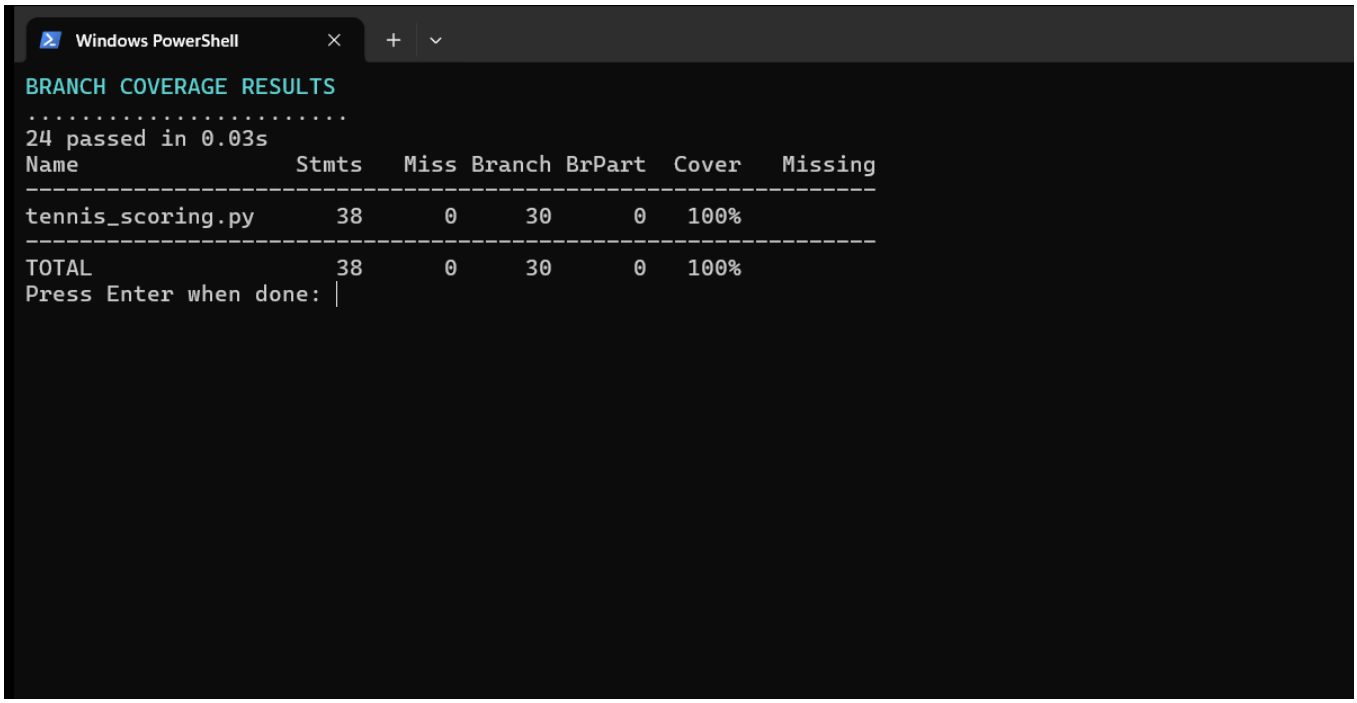
25 tests passed. The report shows 38 of 38 statements covered (100%).



Captured from the PowerShell terminal after the coverage run completed.

Branch Coverage Result

24 tests passed. The report shows 30 branches, no partial branches, and 100% coverage.



Captured from the PowerShell terminal after the coverage run completed.

Appendix A - Code Coverage Test File

Source file: `CodeCoverageTest_34_tennis_scoring.py`

```
"""Tests focused on executing every statement in tennis_scoring."""

import pytest

from tennis_scoring import game_score, match_status, set_winner


@pytest.mark.parametrize(
    ("a", "b", "expected"),
    [
        (0, 0, "0-0"),
        (1, 2, "15-30"),
        (2, 1, "30-15"),
        (3, 2, "40-30"),
        (2, 3, "30-40"),
        (3, 3, "deuce"),
        (4, 3, "advantage A"),
        (3, 4, "advantage B"),
        (5, 3, "game A"),
        (3, 5, "game B"),
        (4, 0, "game A"),
        (0, 4, "game B"),
    ],
)
def test_game_score_statements(a, b, expected):
    assert game_score(a, b) == expected


@pytest.mark.parametrize("a,b", [(-1, 0), (0, -1)])
def test_game_score_rejects_negative_points(a, b):
    with pytest.raises(ValueError, match="negative points"):
        game_score(a, b)


@pytest.mark.parametrize(
    "a,b,expected",
    [
        (6, 0, "A"),
        (0, 6, "B"),
        (7, 6, "A"),
        (6, 7, "B"),
        (6, 5, None),
        (5, 6, None),
        (4, 4, None),
    ],
)
def test_set_winner_statements(a, b, expected):
    assert set_winner(a, b) == expected


@pytest.mark.parametrize(
    "sets_a,sets_b,best_of,expected",
    [
        (2, 0, 3, "A wins"),
        (0, 2, 3, "B wins"),
        (1, 1, 3, "decider"),
        (1, 0, 3, "in progress"),
    ],
)
def test_match_status_statements(sets_a, sets_b, best_of, expected):
    assert match_status(sets_a, sets_b, best_of) == expected
```

Appendix B - Branch Coverage Test File

Source file: `BranchCoverageTest_34_tennis_scoring.py`

```
"""Tests covering both outcomes of decisions in tennis_scoring."""

import pytest

from tennis_scoring import game_score, match_status, set_winner


@pytest.mark.parametrize(
    "a,b,expected",
    [
        (0, 0, "0-0"),
        (1, 2, "15-30"),
        (3, 2, "40-30"),
        (2, 3, "30-40"),
        (3, 3, "deuce"),
        (4, 3, "advantage A"),
        (3, 4, "advantage B"),
        (5, 3, "game A"),
        (3, 5, "game B"),
        (4, 0, "game A"),
        (0, 4, "game B"),
    ],
)
def test_game_score_branches(a, b, expected):
    assert game_score(a, b) == expected


@pytest.mark.parametrize("a,b", [(-1, 0), (0, -1)])
def test_game_score_negative_input_branches(a, b):
    with pytest.raises(ValueError, match="negative points"):
        game_score(a, b)


@pytest.mark.parametrize(
    "a,b,expected",
    [
        (6, 0, "A"),
        (0, 6, "B"),
        (7, 6, "A"),
        (6, 7, "B"),
        (6, 5, None),
        (5, 6, None),
        (4, 4, None),
    ],
)
def test_set_winner_branches(a, b, expected):
    assert set_winner(a, b) == expected


@pytest.mark.parametrize(
    "sets_a,sets_b,best_of,expected",
    [
        (2, 0, 3, "A wins"),
        (0, 2, 3, "B wins"),
        (1, 1, 3, "decider"),
        (0, 0, 3, "in progress"),
    ],
)
def test_match_status_branches(sets_a, sets_b, best_of, expected):
    assert match_status(sets_a, sets_b, best_of) == expected
```